

OOPS

1. Class & Object :-

Class & Object

- * Blue Print - we can relate with our school exam blue print
- * group of similar entities
- * Logical component and not the physical entity
- * Example : phone is a class, object are Samsung Galaxy, Vivo V9, iPhone, and etc
- * Class properties can be Price, screen size, battery capacity, OS type, and etc
- Each object has its own properties
- iPhone 8 properties are Price = 60,000, Battery Capacity = 1,821 mAh, OS = iOS 11
- Vivo V9 Price = 17,000, Battery capacity, 3260 OS = android 8.1

2. What is a Programming Paradigm?

Programming Paradigms

- A programming paradigm is a style, or "way", of programming
- A paradigm is a way of doing something (like programming), not a concrete thing (like a language).

Some Common Programming Paradigms:-

- Imperative : Programming with an explicit sequence of commands that update state.
- Declarative : Programming by specifying the result you want, not how to get it.
- Structured : Programming with clean, goto-free, nested control structures.
- Procedural : Imperative programming with procedure calls.
- Object - Oriented.

3. Imperative Programming Paradigm :-

Imperative Programming Paradigm

- The word "imperative" comes from the Latin "impero" meaning "I command".
- Imperative programming is a paradigm of programming in which the program describes a sequence of steps that change the state of the computer.
- It tells the computer "how" to accomplish a task.
- Programs written this way often compile to binary executables that ~~run~~ more efficiently since all CPU instructions are themselves imperative statements.
- To make programs simpler to understand, statements have been grouped into sections using blocks.

• Procedural and Object-oriented Programming belong under imperative paradigm like C, C++, C#, PHP, Java and Assembly.

4. Declarative Programming Paradigm:-

Declarative Programming Paradigm

- Declarative Programming is a programming Paradigm in which the programmer defines what needs to be accomplished by the program without defining how it needs to be implemented.
- Imperative Programming is like how you do something, and declarative programming is more like what you do, or something.
- Logic, functional and domain-specific languages belong under declarative paradigms. Examples would be HTML, XML, CSS, SQL, Prolog, Haskell, F# and Lisp.
- Declarative code focuses on building logic of software without actually describing its flow.
- For example with HTML you use `img src="image.jpg" />` to tell browser to display an image and you don't care how it does that.

5. Structured Programming Paradigm:-

Structured Programming Paradigm

- Structured programming frequently

employs a top-down design model.

- Structured Programming was first suggested by Corrado Bohm and Giuseppe Jacopini. The two mathematicians demonstrated that any computer program can be written with just three structures: decisions, sequences, and loops.

- Structured Programming is a programming paradigm aimed at improving the clarity, quality, and development time of a computer program by making extensive use of the structured control flow constructs of selection (if/then/else) and repetition (while and for), block structures, and subroutines.

- Most of the programming language uses structured programming language features such as ALGOL, Pascal, PL/I, Ada, C etc.

6. Structured vs Unstructured Programming Paradigm

What is difference between structured and unstructured programming languages?

- Structured and unstructured programming language allow a programmer to divide the whole program into smaller modules. But in unstructured programming language, the whole program must be written in single continuous way.
- Structured programming language is a precursor to Object oriented programming.

(loop) language. But another one is not.

- Structured Programming language produces readable code while Unstructured programming language produces hardly readable code "spaghetti".
- Structured Programming language is easy to modify and debug, while unstructured programming language is very difficult to modify and debug.
- Examples of structured programming languages are C, C++, C#, Java, PERL, Ruby, PHP, ALGOL, Pascal, PL/I and Ada; Examples of unstructured programming language are JESS, FOCAL, MUMPS, TELCOMP, COBOL.

7: Why C language is called a structured programming language?

Why C - Language is called as structured programming language?

- C - language divide the complex problem into smaller modules called functions or Procedure.
- Each module handles a particular job.
- Example of Matrix subtraction program, which is divided into these sub procedures - input matrix, display matrix, subtract matrix, save result matrix to file.
- One major drawback of C language is that similar functions cannot be grouped inside a module or class.

- Date and methods and not be bind together in a module.
- polymorphism and inheritance are not available.

f. Class and Object Program in Java :-

How does a class & object look in Java?

Class

Class phone

```
{
    // Data members
    String OS;
    double Price;
    int batteryCapacity;
}
```

// constructor

```
Public phone (String OS, double Price, int batteryCapacity)
```

```
{
    this.OS = OS;
    this.Price = Price;
    this.batteryCapacity = batteryCapacity;
}
```

// Functions

```
Public double get Price()
```

```
{
    // ...
}

Public int get batteryCapacity()
{
    // ...
}
```

object

```
Public static void main
(String[] args)
{
    Phone iphone = new Phone ("ios",
    60000, 1821);
    Phone vivo = new phone ("
    Android 8.1", 11000, 3620);
}
```


9. polymorphism in OOPS:-

Polymorphism

Function to take on multiple forms

Types of polymorphism:-

- Method overloading (compile time polymorphism) - same method name but with different operations.

Example: `add (int a, int b)` add two numbers,
`add (int a, int b, int c)` add three numbers.

- Method overriding (Run-time polymorphism) - if there are 2 classes called parents and children. Parents class is a base class and children is a child class. children class inherits the parents class. Both the classes have `get Property Val` method.

Advantages:

Single name can be used for the same type of methods.

Real-time Example:

In school behave like a student

In Market behave like a customer

In office behave like a professional

10. Encapsulation in OOPS:-

Encapsulation

- wrapping up of data and methods into a single unit.

- To avoid the access of private data members from outside the class.

You can get a Coco cola which is inside the machine only by inserting the Coin in the Vending machine. Here, Coco-Cola is the private data of that vending machine. You can access that using the operations provided by that Vending machine class.

Advantages:-

- Data Hiding - Restricts the access of some data from outside world directly.
- flexibility - Allows whether the data should be read - only or write-read.

Real - life example:-

Consider a Coco-cola vending machine. Here, both data (Coco-cola bottles) and operations (buy Coco Cola()) are wrapped in a vending machine. We can treat that vending machine as a class.

11. Inheritance in OOPS:-

Inheritance

- Inheritance the properties from something
- OOPS concept in which one object acquires the properties and behavior

of the parent Object. :- Aggregation

- Parent-child relationship between two classes.

Advantages:-

- Reusability - can reduce duplication of the same code.
- Extensibility - extend the base class logic as per business logic of the derived class.
- Data hiding - base class can decide which Properties need to be inherited.

Real time example:-

Humans inherit Properties from our parents directly and from our grandparents indirectly.

- Grand Parents property value = 10 lakhs
- Grand Parents Property Value = 10 lakhs
Parents Property Value = 20 lakhs
Total Property Value = 30 lakhs
- Grand Parents Property Value = 10 lakhs
Parent Property value = 20 lakhs
Children Property Value = 50 lakhs
Total Property Value = 80 lakhs

12. Abstractions in OOPS:-

Abstraction

- Exposing only the required things to the outside world while hiding the inner details.

Advantages:-

- Abstraction makes the code as readable and more simpler. Because the complexity of code will be hidden.
- It reduces the code because it hides the unnecessary things.

Real - life Example:-

We use Air-Conditioner by switching on and off using remote but we don't care about the inner working mechanism of Air-conditioner. We are only concerned about the final output.